МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
Факультет прикладной математики и информатики
Кафедра многопроцессорных систем и сетей

Сергиенко Лев Эдуардович

Отчет по “Лабораторная работа 6”

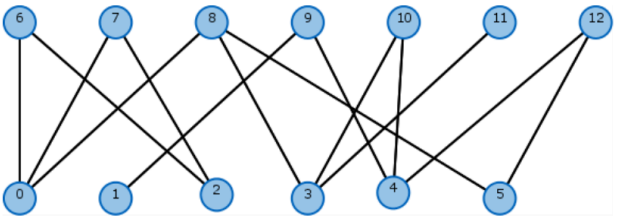
Преподаватели
Волчецкая П.С.
Дрепакова А.В.

Минск, 2025

Формулировка проблемы

Решите две задачи.
1. Найдите максимальное паросочетание и минимальное вершинное покрытие в двудольном графе.
2. Решите задачу о назначениях:

Вариант 21.



8	4	5	6	2
7	6	4	6	8
7	8	2	4	4
5	3	7	2	3
7	8	9	8	2

Математическая модель

Задача 1

Граф: $G = (L \cup R, E)$, где:

- $L = \{0, 1, 2, 3, 4, 5\}$ — левая доля (6 вершин)
- $R = \{6, 7, 8, 9, 10, 11, 12\}$ — правая доля (7 вершин)
- E — множество рёбер, заданное матрицей смежности

Паросочетание: $M \subseteq E$ такое, что никакие два ребра не имеют общей вершины.

Вершинное покрытие: $C \subseteq L \cup R$ такое, что каждое ребро инцидентно хотя бы одной вершине из C.

Теорема Кёнига: В двудольном графе размер максимального паросочетания равен размеру минимального вершинного покрытия: $|M_{max}| = |C_{min}|$

Алгоритм решения

Алгоритм Куна для нахождения максимального паросочетания:

1. Инициализация: matchL[v] = -1 для всех вершин левой доли
2. Для каждой вершины $v \in L$:
    - Пытаемся найти увеличивающую цепь с помощью DFS
    - Если цепь найдена, увеличиваем паросочетание

Функция поиска увеличивающей цепи:

```
kuhn(v, used):
  если used[v]: return False
  used[v] = True
  для каждой смежной вершины u:
    если u свободна ИЛИ можно переназначить пару u:
      назначаем v ↔ u
      return True
  return False
```

Алгоритм нахождения минимального вершинного покрытия (на основе теоремы Кёнига):

1. Строим остаточный граф из ненасыщенных вершин левой доли
2. Выполняем DFS, помечая достижимые вершины
3. Вершинное покрытие = (непомеченные левые) $\cup$ (помеченные правые)

Асимптотическая сложность:

- Алгоритм Куна: $O(V \cdot E)$, где V — число вершин, E — число рёбер

Задача 2

Требуется найти перестановку $(p_1, p_2, p_3, p_4, p_5)$, минимизирующую суммарную стоимость:

$$\min \sum_{i=1}^{5} A[i-1][p_i - 1]$$

при условии, что каждый работник выполняет ровно одну работу, и каждую работу выполняет ровно один работник.

Переменные: Перестановка $\pi: \{1,2,3,4,5\} \to \{1,2,3,4,5\}$

Целевая функция:

$$\#\#(\pi) = \sum_{i=1}^{5} c_{i,\pi(i)} \to \min$$

где $c_{i,j}$ — стоимость назначения работника i на работу j.

Алгоритмы решения

Метод 1: Полный перебор

Перебираем все n! перестановок и выбираем минимальную.

Асимптотическая сложность: O(n! · n)

Метод 2: Венгерский алгоритм

Оптимальный алгоритм для задачи о назначениях, основанный на двойственности линейного программирования.

Идея алгоритма:

1. Вводим потенциалы u_i для работников и v_j для работ
2. Поддерживаем допустимость: $u_i + v_j \leq c_{i,j}$ для всех (i,j)
3. Строим паросочетание на рёбрах равенства: $u_i + v_j = c_{i,j}$
4. Итеративно улучшаем решение с помощью увеличивающих цепей

Основные шаги:

1. Для каждого работника i находим минимальную стоимость по строке
2. Строим остаточный граф с учётом потенциалов
3. Используем алгоритм Куна для построения совершенного паросочетания
4. Пересчитываем потенциалы до получения оптимального решения

Асимптотическая сложность: $O(n^3)$

Решение

```
from itertools import permutations

A = [
```

```
    [1, 1, 1, 0, 0, 0, 0],
    [0, 0, 0, 1, 0, 0, 0],
    [1, 1, 0, 0, 0, 0, 0],
    [0, 0, 1, 0, 1, 1, 0],
    [0, 0, 0, 1, 1, 0, 1],
    [0, 0, 1, 0, 0, 0, 1],
]


n = len(A)  # нижних = 6 (левая часть)
m = len(A[0])  # верхних = 7 (правая часть)
adj = [[j for j, val in enumerate(row) if val] for row in
A]

matchL = [-1] * n
matchR = [-1] * m


def kuhn(v, used):
    if used[v]:
        return False
    used[v] = True
    for u in adj[v]:
        if matchR[u] == -1 or kuhn(matchR[u], used):
            matchR[u] = v
            matchL[v] = u
            return True
    return False


for v in range(n):
    used = [False] * n
    kuhn(v, used)

matching = [(v, n + matchL[v]) for v in range(n) if
matchL[v] != -1]
```

```
print("Паросочетание (нижняя -> верхняя):")
for a, b in matching:
    print(f"{a} -- {b}")
print("Размер паросочетания:", len(matching))

# --- находим минимальное вершинное покрытие по теореме Кёнига ---
L_vis = [False] * n
R_vis = [False] * m


def dfs_from_left(v):
    L_vis[v] = True
    for u in adj[v]:
        if not R_vis[u] and matchL[v] != u:
            R_vis[u] = True
            if matchR[u] != -1 and not L_vis[matchR[u]]:
                dfs_from_left(matchR[u])


for v in range(n):
    if matchL[v] == -1 and not L_vis[v]:
        dfs_from_left(v)

min_vertex_cover_left = [v for v in range(n) if not L_vis[v]]
min_vertex_cover_right = [n + u for u in range(m) if R_vis[u]]

min_vertex_cover = min_vertex_cover_left + min_vertex_cover_right

print("\nМинимальное вершинное покрытие (индексы: левые 0..{}, правые {}..{}):".format(n - 1, n, n + m - 1))
print("Левые вершины в покрытии:", min_vertex_cover_left)
```

```
print("Правые вершины в покрытии со сдвигом:",
min_vertex_cover_right)
print("Общее покрытие:", min_vertex_cover)
print("Размер покрытия:", len(min_vertex_cover))

##############################################

A = [
    [8, 4, 5, 6, 2],
    [7, 6, 4, 6, 8],
    [7, 8, 2, 4, 4],
    [5, 3, 7, 2, 3],
    [7, 8, 9, 8, 2],
]
n = len(A)
best = min(((sum(A[i][p[i]] for i in range(n)), p) for p
in permutations(range(n))), key=lambda x: x[0])
cost, p = best

print()
print("Перестановка:", tuple(i + 1 for i in p))
print("Минимальная стоимость:", cost)

##############################################


def hungarian(a):
    n = len(a)
    m = len(a[0])
    u = [0] * (n + 1)
    v = [0] * (m + 1)
    p = [0] * (m + 1)
    way = [0] * (m + 1)
    for i in range(1, n + 1):
        p[0] = i
        j0 = 0
```

```
        minv = [float("inf")] * (m + 1)
        used = [False] * (m + 1)
        while True:
            used[j0] = True
            i0 = p[j0]
            delta = float("inf")
            j1 = 0
            for j in range(1, m + 1):
                if not used[j]:
                    cur = a[i0 - 1][j - 1] - u[i0] - v[j]
                    if cur < minv[j]:
                        minv[j] = cur
                        way[j] = j0
                    if minv[j] < delta:
                        delta = minv[j]
                        j1 = j
            for j in range(0, m + 1):
                if used[j]:
                    u[p[j]] += delta
                    v[j] -= delta
                else:
                    minv[j] -= delta
            j0 = j1
            if p[j0] == 0:
                break
        while True:
            j1 = way[j0]
            p[j0] = p[j1]
            j0 = j1
            if j0 == 0:
                break

ans = [-1] * n
for j in range(1, m + 1):
    if p[j] != 0:
        ans[p[j] - 1] = j - 1
```

```
    cost = sum(a[i][ans[i]] for i in range(n))
    return cost, ans


A = [
    [8, 4, 5, 6, 2],
    [7, 6, 4, 6, 8],
    [7, 8, 2, 4, 4],
    [5, 3, 7, 2, 3],
    [7, 8, 9, 8, 2],
]

cost, assignment = hungarian(A)

print()
print("Перестановка:", tuple(x + 1 for x in assignment))
print("Минимальная стоимость:", cost)
```

Результат

Паросочетание (нижняя -> верхняя):
0 — 7
1 — 9
2 — 6
3 — 10
4 — 12
5 — 8
Размер паросочетания: 6

Минимальное вершинное покрытие (индексы: левые 0..5, правые 6..12):
Левые вершины в покрытии: [0, 1, 2, 3, 4, 5]
Правые вершины в покрытии со сдвигом: []
Общее покрытие: [0, 1, 2, 3, 4, 5]
Размер покрытия: 6

Перестановка: (2, 1, 3, 4, 5)
Минимальная стоимость: 17

Перестановка: (2, 1, 3, 4, 5)
Минимальная стоимость: 17